

CS102 – Algorithms and Programming II
Programming Assignment 2
Fall 2025-2026

ATTENTION:

- Compress all of the Java program source files (.java) files into a single zip file.
- The name of the zip file should follow the below convention, where you replace the variables “Sec1”, “Surname” and “Name” with your actual section, surname and name:
CS102_Sec1_Asst2_Surname_Name.zip
- Upload the above zip file to Moodle by the deadline (if not, significant points will be taken off). You will get a chance to update and improve your solution by consulting to the TAs and tutors during the lab. You have to make the preliminary submission before the lab day. After the TA checks your work in the lab, you will make your final submission. Even if your code does not change in between these two versions, you should make two submissions for each assignment.

GRADING WARNING:

- Please read the grading criteria provided on Moodle. The work must be done individually. Code sharing is strictly forbidden. We are using sophisticated tools to check the code similarities. The Honor Code specifies what you can and cannot do. Breaking the rules will result in disciplinary action.

ANALYZING WORD FREQUENCIES IN A TEXT FILE

For this lab assignment, you will implement a Java console application that analyzes the unique words' letter frequencies in a given text file. Your application should ask for the name of the text file in your project folder. You will read the text line by line and extract the individual words of the text using either the split method of the String class or a StringTokenizer.

Reading from a text file requires using the Scanner with a File input. The file should be in your project folder so your application can find it. You may use the following code segment to read the text file:

```
String filename = "words.txt";
Scanner sc = new Scanner(new File(filename));
while(sc.hasNextLine()) {
    String line = sc.nextLine();
    System.out.println(line);
}
sc.close();
```

You will see that trying to open a file with "new File(filename)" results in a syntax error with the message indicating an unhandled exception. You will learn more about exceptions later in the course. For now, you should know that using a File can result in an exception if the specified file is not included in the project folder; therefore, you need to handle this situation. If you include the "throws FileNotFoundException" declaration in the method signatures as the IDE suggests, this will leave handling the exception to the classes that call this method. Alternatively, surround the code that performs the file reading with a try-catch block. Both approaches are accepted for the assignment.

You should implement a WordList class to keep the words you read from the text file. This class should have an addWord(String wordText) method to include the words you extract from the file.

This method should make sure that the inserted words do not have duplicates. For example, if the word "apple" is already in our WordList, we should not add it again; the addWord method should do nothing if the given word is already in the list. You may use an ArrayList or any other Java Collections object to keep your word Strings in it; you are also welcome to use a standard String array.

Include a printExamples method in your WordList to print a random set of ten words on screen so that you can check if you are reading the words correctly. This method should print a different random set each time we call it. You may determine a random index to start, then print the ten consecutive words in your array, separated by comma symbols between each word. Your WordList should also include analysis methods to perform the following operations:

1. The unique letter occurrence ratio: Calculate the number of occurrences in your words for each letter of the English alphabet. Suppose our WordList contains the words "apple, dog, food", the ratio of "a" is 1/3, and the ratio of "o" is 2/3. That is, out of the three words we have, two of them contain at least one occurrence of the letter "o." If a word has multiple instances of the same letter, it is only counted once.
2. The word length ratio: Based on the longest word in your WordList, calculate the number of occurrences of each word length. Suppose we have "to, dog, cat, bird" in our WordList, then we have 0/4 one-letter words, 1/4 two-letter words, 2/4 three-letter words, and 1/4 four-letter words. Note that we do not calculate the ratio for word lengths higher than four, as our list includes four-letter words, the highest.
3. Proceeding letter ratio: For each letter, calculate the ratio for the following letter. For example, for the letter "a", among the cases where there is a pair such as "a_", what is the ratio where we have "aa", "ab", "ac", and so on? Suppose our list contains "to, dog, food, foot", then for the letter o, among the pairs such as "o_", we have "og" 1/5 of the time, "oo" as 2/5, and "ot" as 1/5, and finally "od" as 1/5. Note that, since in the word "to", the character "o" is not followed by a different character, it is not counted as part of our proceeding letter ratio. This ratio should be calculated for all such pairs in our WordList.

After running the first option, the program should display the occurrence ratio for each alphabet letter. Running the second option should display all the word length ratios different from zero. The third option should not print out the complete results, as the screen would be too crowded. Instead, let the user input a character, then the program should print out all the non-zero following letter ratios for that letter.

Preliminary Submission: You will submit an early version of your solution before the final submission. This version should at least include the following:

The methods to read the text file and store its unique words should be complete.

Not completing the preliminary submission on time results in 50% reduction of this assignment's final grade.